# Avaliacao justa em previsao de demanda: baselines, IA classica e Reservoir Computing

***Resumo.*** *Este artigo organiza o benchmark central da serie. O objetivo nao e apenas mostrar uma tabela final, mas ensinar como comparar modelos de series temporais de forma justa quando o interesse e chegar ate RC e QRC sem perder contato com tecnicas classicas fortes. Por isso, o texto formaliza o protocolo experimental, resume as implementacoes ja construidas no projeto, apresenta as equacoes minimas das familias avaliadas e discute o benchmark consolidado real do recorte Favorita. O resultado mais importante e metodologico:* `ETS`*,* `XGBoost` *e* `Prophet` *formam uma referencia forte que impede avaliar RC e QRC contra baselines fracos.*

## 0.1. 1. O que o leitor vai aprender

Ao final deste artigo, voce sera capaz de:

1. comparar modelos temporais com um protocolo realmente comum;
2. entender o papel de cada familia de modelo no benchmark;
3. ler uma tabela comparativa sem cair em conclusoes apressadas;
4. reproduzir o benchmark com os scripts do projeto;
5. usar os resultados classicos como referencia seria para RC e QRC.

## 0.2. 2. Por que avaliacao em series temporais e delicada

Em series temporais, avaliacao injusta aparece facilmente:

- quando um modelo usa um split temporal diferente;
- quando features usam informacao do futuro;
- quando um modelo recebe regressoras exogenas e outro nao;
- quando os horizontes de previsao nao coincidem;
- quando as metricas destacadas mudam de um experimento para outro.

Para evitar isso, este projeto fixa o seguinte principio:

$$\text{mesmo recorte} + \text{mesmo split} + \text{mesmas metricas} = \text{comparacao honesta.}$$

## 0.3. 3. Protocolo experimental da serie

### 0.3.1. 3.1 Recorte comum

Todos os modelos usam:

- `store_nbr = 1`
- `family = BEVERAGES`
- frequencia diaria
- ultimos `90` dias como teste

### 0.3.2. 3.2 Regra de split

Formalmente:

$$\mathcal{D}_{train} = \{(x_t, y_t)\}_{t=1}^{T-90}, \qquad \mathcal{D}_{test} = \{(x_t, y_t)\}_{t=T-89}^{T}.$$

### 0.3.3. 3.3 Modelos avaliados

| Modelo | Implementacao | Ideia central |
|---|---|---|
| Seasonal Naive | `code/baselines/seasonal_naives/` | repete a ultima semana observada |
| ETS | `code/baselines/ets/` | modelo de nivel, tendencia e sazonalida |
| Prophet | `code/baselines/prophet/` | decomposicao de tendencia, sazonalidad |
| XGBoost | `code/classical/xgboost/` | aprendizado tabular sobre lags e medias |
| LSTM | `code/classical/lstm/` | rede recorrente treinada por gradiente |
| RC | `code/rc/` | reservatorio fixo e readout linear |
| QRC | `code/qrc/` | reservatorio quantico implementado con |

### 0.3.4. 3.4 Metricas avaliadas

O benchmark usa:

$$\text{MAE}, \ \text{RMSE}, \ \text{WAPE}, \ \text{sMAPE}.$$

As implementacoes dessas metricas estao em `code/common/metrics.py`.

## 0.4. 4. As familias de modelo em uma pagina

Para manter a comparacao didatica, vale resumir a logica de cada familia.

### 0.4.1. 4.1 Seasonal Naive

$$\hat{y}_t = y_{t-7}.$$

Serve como baseline sazonal minimo.

### 0.4.2. 4.2 ETS

O ETS aditivo pode ser resumido por

$$\begin{aligned}
\ell_t &= \alpha(y_t - s_{t-m}) + (1-\alpha)(\ell_{t-1} + b_{t-1}), \\
b_t &= \beta(\ell_t - \ell_{t-1}) + (1-\beta)b_{t-1}, \\
s_t &= \gamma(y_t - \ell_t) + (1-\gamma)s_{t-m}.
\end{aligned}$$

### 0.4.3. 4.3 Prophet

O Prophet modela a serie como

$$y(t) = g(t) + s(t) + h(t) + \beta x_t + \epsilon_t,$$

em que `onpromotion` entra como regressor exogeno no projeto.

### 0.4.4. 4.4 XGBoost

O modelo tabular usa lags, medias moveis e calendario para aprender

$$\hat{y}_t = \sum_{m=1}^{M} f_m(z_t),$$

com $z_t$ contendo `lag_1`, `lag_7`, `lag_14`, `lag_28`, medias moveis e calendarios.

### 0.4.5. 4.5 LSTM

A LSTM usa portas para atualizar memoria:

$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i), \quad f_t = \sigma(W_f[h_{t-1}, x_t] + b_f),$$
$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t.$$

### 0.4.6. 4.6 RC

$$x_t = (1-\alpha)x_{t-1} + \alpha \tanh(W_{res}x_{t-1} + W_{in}u_t + b).$$

### 0.4.7. 4.7 QRC

O QRC sera detalhado nos artigos 6 e 7, mas aqui ele entra como

$$z_t^{(j)} = \langle \psi_t | O_j | \psi_t \rangle, \qquad \hat{y}_t = W_{out}[1; u_t; z_t].$$

## 0.5. 5. Benchmark comparativo atual

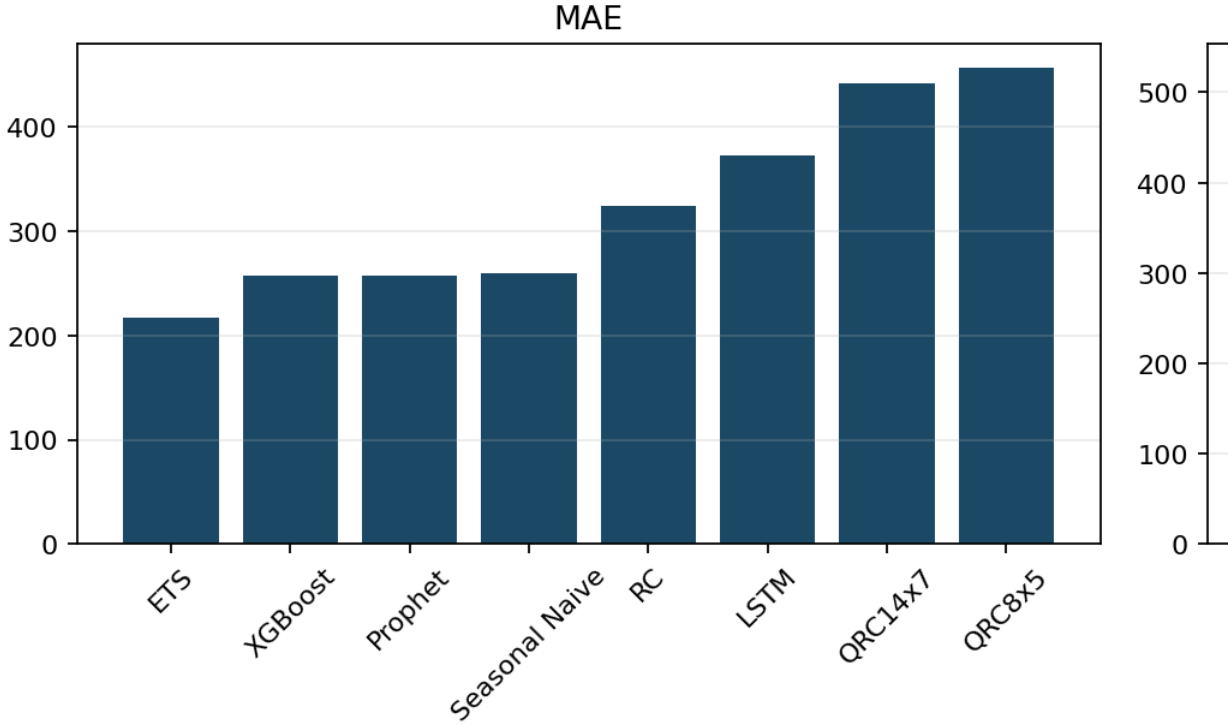
A tabela abaixo resume os resultados reais obtidos no projeto.

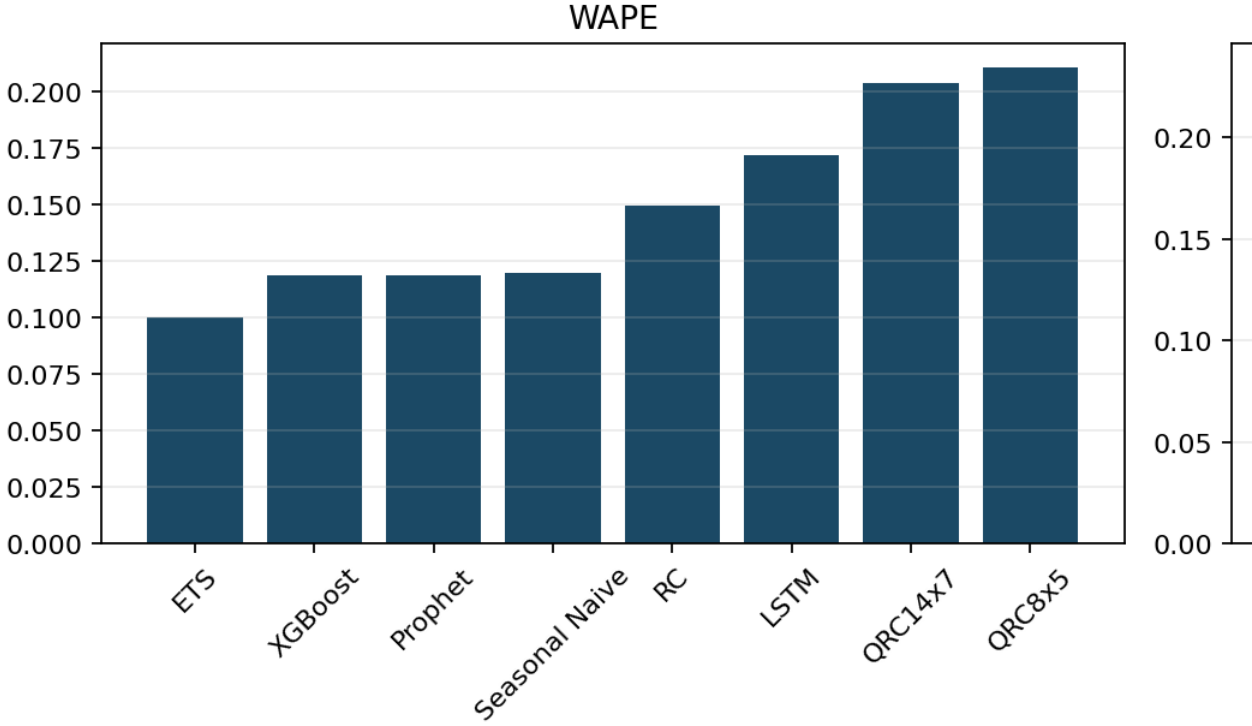
| Modelo | Familia | MAE | RMSE | WAPE | sMAPE |
|---|---|---|---|---|---|
| ETS | baseline estatistico | 216.303 | 307.638 | 0.0999 | 0.1042 |
| XGBoost | ML classico | 256.666 | 332.828 | 0.1185 | 0.1237 |
| Prophet | baseline estatistico | 257.584 | 321.818 | 0.1189 | 0.1319 |

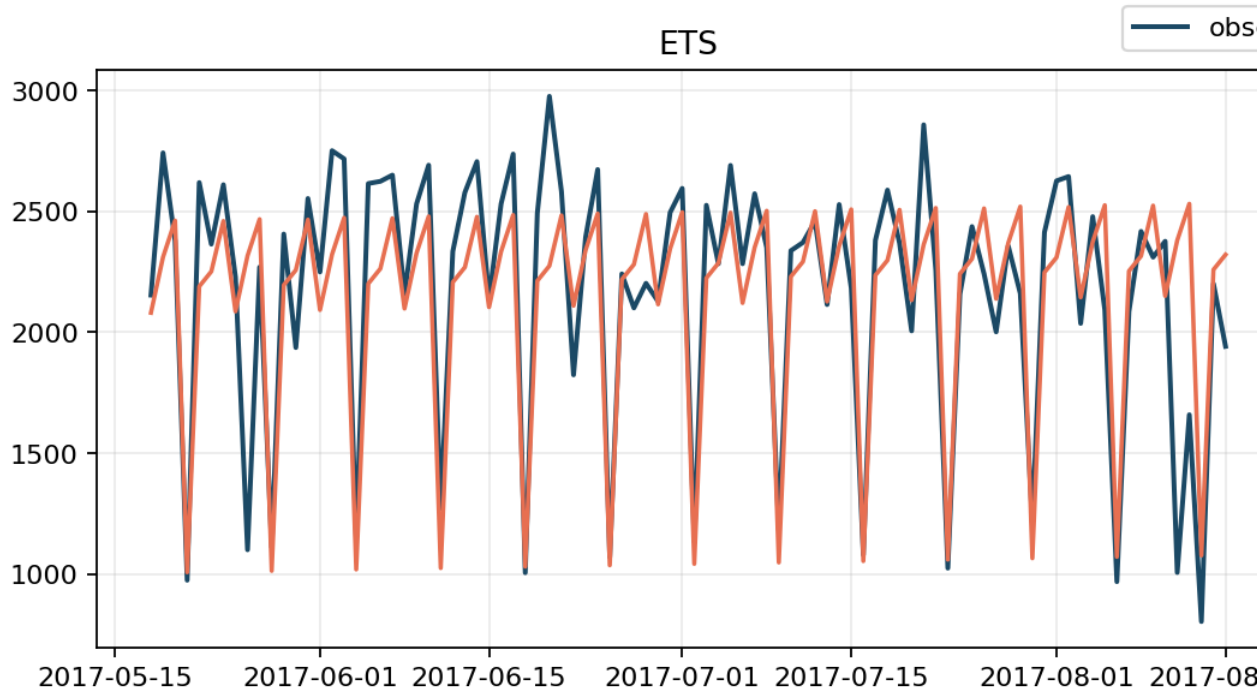
| Modelo | Familia | MAE | RMSE | WAPE | sMAPE |
|---|---|---|---|---|---|
| Seasonal Naive | baseline estatistico | 259.067 | 352.280 | 0.1196 | 0.1361 |
| RC | reservoir computing classico | 324.294 | 423.417 | 0.1498 | 0.1653 |
| LSTM | DL classico | 371.827 | 461.342 | 0.1717 | 0.1822 |
| QRC14x7 | reservoir computing quantico | 441.286 | 525.325 | 0.2038 | 0.2269 |
| QRC8x5 | reservoir computing quantico | 455.833 | 526.949 | 0.2105 | 0.2342 |

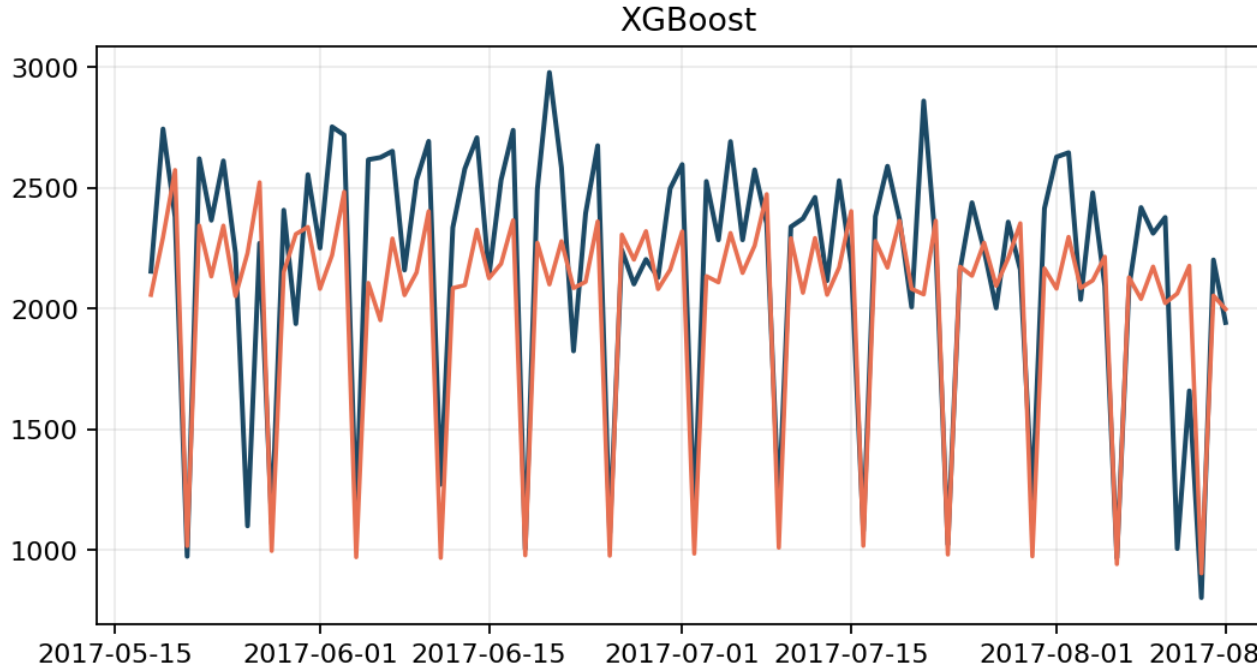
Uma leitura ordenada por rank medio reforca a comparacao.

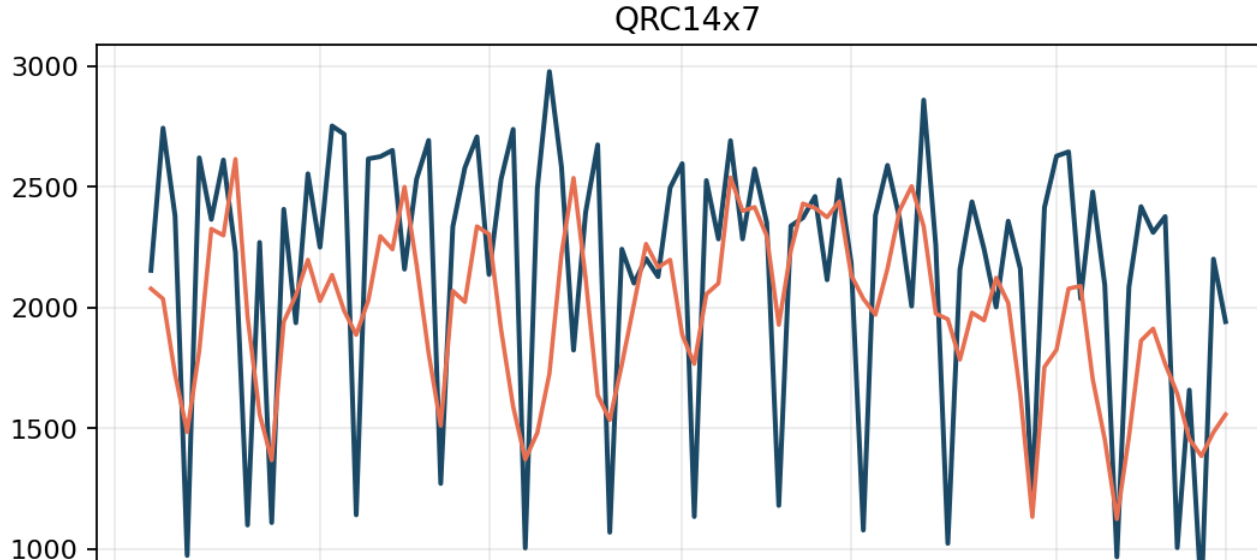
| Modelo | Rank medio | MAE | RMSE | WAPE | sMAPE |
|---|---|---|---|---|---|
| ETS | 1.00 | 216.303 | 307.638 | 0.0999 | 0.1042 |
| XGBoost | 2.25 | 256.666 | 332.828 | 0.1185 | 0.1237 |
| Prophet | 2.75 | 257.584 | 321.818 | 0.1189 | 0.1319 |
| Seasonal Naive | 4.00 | 259.067 | 352.280 | 0.1196 | 0.1361 |
| RC | 5.00 | 324.294 | 423.417 | 0.1498 | 0.1653 |
| LSTM | 6.00 | 371.827 | 461.342 | 0.1717 | 0.1822 |
| QRC14x7 | 7.00 | 441.286 | 525.325 | 0.2038 | 0.2269 |
| QRC8x5 | 8.00 | 455.833 | 526.949 | 0.2105 | 0.2342 |


MAE
0
100
200
300
400
500
ETS
XGBoost
Prophet
Seasonal Naive
RC
LSTM
QRC14x7
QRC8x5


WAPE
0.000
0.025
0.050
0.075
0.100
0.125
0.150
0.175
0.200
0.00
0.05
0.10
0.15
0.20
ETS
XGBoost
Prophet
Seasonal Naive
RC
LSTM
QRC14x7
QRC8x5


obs
ETS
3000
2500
2000
1500
1000
2017-05-15
2017-06-01
2017-06-15
2017-07-01
2017-07-15
2017-08-01
2017-08


XGBoost
3000
2500
2000
1500
1000
2017-05-15
2017-06-01
2017-06-15
2017-07-01
2017-07-15
2017-08-01
2017-08


QRC14x7
3000
2500
2000
1500
1000

## 0.6. 6. Leitura dos resultados

### 0.6.1. 6.1 ETS na lideranca

O `ETS` foi o melhor modelo no recorte em todas as metricas principais. Isso e importante porque mostra que um baseline estatistico bem implementado continua sendo uma referencia dificil de bater.

### 0.6.2. 6.2 XGBoost e Prophet como faixa intermediaria forte

`XGBoost` e `Prophet` formam uma segunda faixa de desempenho. Ambos ficaram muito proximos do `Seasonal Naive` em algumas metricas, mas entregaram um pacote mais robusto no conjunto do benchmark.

### 0.6.3. 6.3 O que o RC entrega

O `RC` superou a `LSTM`, o que e pedagogicamente interessante. Em um setup pequeno e pouco ajustado, uma rede recorrente treinada por gradiente nao vence automaticamente um reservatorio fixo.

### 0.6.4. 6.4 O que o QRC entrega

O `QRC` funcionou de ponta a ponta e foi comparado no mesmo recorte de dados. No entanto, ficou abaixo das tecnicas classicas neste benchmark. Isso nao invalida o estudo; ao contrario, torna a comparacao mais honesta.

## 0.7. 7. O que significa uma comparacao justa

Este benchmark so e didaticamente util porque respeita tres regras:

1. todos os modelos usam os mesmos dados;
2. todos produzem previsao para os mesmos `90` dias;
3. todos sao julgados pelas mesmas metricas.

Isso evita a narrativa enganosa de que um modelo "venceu"porque recebeu mais contexto ou uma tarefa mais facil.

## 0.8. 8. Passo-a-passo para reproduzir o benchmark

A reproducao pode ser feita em lotes, executando um script por familia:

```
python code/baselines/seasonal_naives/run.py
python code/baselines/ets/run.py
python code/baselines/prophet/run.py
python code/classical/xgboost/run.py
python code/classical/lstm/run.py
python code/rc/run.py
python code/qrc/run.py
```

Os testes associados seguem a mesma organizacao:

```
pytest code/baselines/seasonal_naives/test_model.py
pytest code/baselines/ets/test_model.py
pytest code/baselines/prophet/test_model.py
pytest code/classical/xgboost/test_model.py
pytest code/classical/lstm/test_model.py
pytest code/rc/test_model.py
pytest code/qrc/test_model.py
pytest code/qrc/test_objective.py
```

### 0.9. 9. Checklist de boas praticas

O artigo deixa um checklist minimo para os proximos experimentos:

- manter o split temporal fixo;
- registrar metricas em arquivo;
- salvar previsoes e figuras;
- documentar hiperparametros;
- nao comparar RC e QRC apenas com baselines fracos.

### 0.10. 10. Regras herdadas para a fase QRC

Quando chegarmos ao QRC, duas regras deste benchmark continuam valendo:

1. o QRC deve ser comparado com as tecnicas classicas ja implementadas;
2. o mesmo recorte de dados deve ser mantido, mesmo que o reservatorio quantico simplifique sua representacao interna.

Isso e o que transforma o estudo final em um artigo de ensino serio, e nao em uma demo isolada.

### 0.11. 11. Conclusao

O benchmark consolidado da serie esta agora estabelecido. Ele mostra que a competicao relevante para RC e QRC nao e contra strawmen, mas contra modelos classicos fortes e bem executados. Isso torna a rota ate QRC intelectualmente mais exigente, mas tambem muito mais valiosa.

### 0.12. Entregaveis associados no repositorio

- benchmark consolidado: `code/RESULTS.md`
- implementacoes: `code/baselines/`, `code/classical/`, `code/rc/`, `code/qrc/`
- artefatos comparativos deste artigo: `computational_results_20260402_222902/`

### 0.13. Referencias


- Hyndman, R. J.; Athanasopoulos, G. Forecasting: Principles and Practice.
- Taylor, S. J.; Letham, B. Forecasting at Scale.
- Chen, T.; Guestrin, C. XGBoost: A scalable tree boosting system.
- Hochreiter, S.; Schmidhuber, J. Long short-term memory.
- Jaeger, H. The "echo state"approach to analysing and training recurrent neural networks.